



ACRON Methodology Part-5

فاز 4: Products Domain



در Part-4 ، فاز 4 تا قدم 56 پیش رفت



مستندسازی نیمه خودکار با استفاده از Swagger

در دنیای Django REST Framework، ما دو انتخاب معروف برای Swagger داریم: `drf-` `yasg` (قدیمی و مبتنی بر OpenAPI 2.0) و `drf-spectacular` (مدرن، قدرتمند و مبتنی بر OpenAPI 3.0).

ما قطعاً مسیر حرفه‌ای‌تر یعنی `drf-spectacular` را انتخاب می‌کنیم.



قدم اول: نصب و تنظیمات پایه

57- نصب پکیج `drf-spectacular` در محیط مجازی:
در ترمینال خود دستور زیر را وارد کنید:

```
pipenv install drf-spectacular
```

58- اضافه کردن به requirements.txt

در ترمینال خود دستور زیر را وارد کنید:

```
pipenv requirements > requirements.txt
```

59- معرفی به جنگو و DRF:

فایل `config/settings/base.py` را باز کنید. ما باید سه تغییر در این فایل ایجاد کنیم:

ابتدا پکیج را به اپلیکیشن‌های نصب‌شده اضافه کنید:

```
] = INSTALLED_APPS
... #
API , 'drf_spectacular'

, 'apps.accounts'
, 'apps.customers'
, 'apps.products'
[
```

60- سپس به DRF بگویید که برای تولید شمای (Schema) مستندات، از کلاس این پکیج استفاده کند:

```
} = REST_FRAMEWORK
# تنظیمات قبلی شما (مثل Authentication و
(Pagination) اینجا می‌مانند...

# اضافه کردن کلاس تولیدکننده مستندات
```

```
DEFAULT_SCHEMA_CLASS': 'drf_spectacula'  
, 'r.openapi.AutoSchema'  
{
```

61- در نهایت، تنظیمات ظاهری و هویتی Swagger را به انتهای فایل `base.py` اضافه کنید:

```
# تنظیمات اختصاصی Swagger  
} = SPECTACULAR_SETTINGS  
, 'TITLE': 'ACRON Project API'  
, 'DESCRIPTION': 'مستندات جامع API های فروش  
گاه آکرون شامل بخش مشتریان و محصولات'  
, 'VERSION': '1.0.0'  
, 'SERVE_INCLUDE_SCHEMA': False'  
# برای تم  
یز ما ندن خروجی نهایی
```

```
# تنظیمات امنیتی برای تست API ها داخل خود  
مرورگر  
, 'SECURITY'  
{ 'jwtAuth' }  
, [  
, 'SECURITY_DEFINITIONS'  
, 'jwtAuth'  
, 'type': 'http'  
, 'scheme': 'bearer'  
, 'bearerFormat': 'JWT'  
{  
{  
{
```

کالبدشکافی تنظیمات (بدون Vibe Coding)

- چرا `DEFAULT_SCHEMA_CLASS` را تغییر دادیم؟ DRF به صورت پیش فرض یک سیستم تولید شمای ساده دارد که خیلی امکانات جالبی ندارد. با این خط، ما "مغز" تحلیلگر DRF را با "مغز" قدرتمند `drf-spectacular` جایگزین می‌کنیم تا بتواند روابط پیچیده (مثل گالری تصاویر و JWT) را بفهمد.
- بخش `SECURITY_DEFINITIONS` چیست؟ یکی از جذاب‌ترین امکانات Swagger این است که دکمه‌ای به نام `Try it out` دارد. اما API‌های پروفایل مشتری ما (مثل `GET //api/customers/me`) قفل هستند و توکن می‌خواهند. این تنظیمات به Swagger UI می‌فهماند که پروژه ما از `Bearer Token` استفاده می‌کند. با این کار، یک دکمه  (Authorize) بالای صفحه Swagger ظاهر می‌شود که می‌توانید توکن خود را آنجا قرار دهید و مستقیماً از داخل مرورگر API‌ها را تست کنید.



قدم دوم: مسيردهی (Routing) URL‌ها

حالا باید URL‌هایی بسازیم که فرانت‌اند بتواند از طریق آن‌ها فایل مستندات و رابط کاربری Swagger را ببیند.

62- فایل اصلی `config/urls.py` را باز کنید و کدهای زیر را اضافه کنید:

```
from django.contrib import admin
from django.urls import path, include
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView, SpectacularRedocView

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('apps.api.urls'))

    ----- Swagger URLs ----- #
    # -----
    # ۱. تولید فایل خام OpenAPI (به فرمت YAML)
```


(L/JSON

```
path('api/schema/', SpectacularAPIView.a  
,s_view(), name='schema')
```

۲. رابط کاربری گرافیکی Swagger (توسعه د
هنگام بک اند و فرانت اند)

```
path('api/docs/', SpectacularSwaggerVie  
w.as_view(url_name='schema'), name='swagger-  
,ui')
```

۳. رابط کاربری Redoc (جایگزین Swagger،
مناسب برای ارائه به مدیران)

```
path('api/redoc/', SpectacularRedocView.  
,as_view(url_name='schema'), name='redoc')  
[
```



کالبدشکافی URL های مستندات

- **SpectacularAPIView** (`/api/schema/`): وقتی این URL این مسیر هیچ گرافیکی ندارد. وقتی این (`/api/schema/`) **SpectacularAPIView** ها شما را می خواند و یک فایل Model ها و Serializers، Viewها صدا می زنید، پکیج تمام شماست API "تولید می کند. این فایل "نقشه راه (YAML معمولاً) متنی ماشین خوان
- **SpectacularSwaggerView** (`/api/docs/`): این همان رابط کاربری جذابی است که توسعه دهندگان دوستش دارند. این صفحه در واقع فایل خام مرحله قبل را می خواند و آن را به دکمه ها و فرم های تعاملی تبدیل می کند (`url_name='schema'`)
- **SpectacularRedocView** (`/api/redoc/`): گرافیک متفاوت و کلاسیک تری نسبت Redoc ابزار (`/api/redoc/`) **SpectacularRedocView** دارد (بدون قابلیت تست تعاملی). بسیاری از شرکت های بزرگ مستندات Swagger به منتشر می کنند، چون خواندن آن مانند یک کتاب مرتب است Redoc عمومی خود را با



اجرای نهایی و تست

حالا سرور پروژه را روشن کنید:

```
python manage.py runserver
```

مرورگر خود را باز کنید و به آدرس زیر بروید:

[/http://127.0.0.1:8000/api/docs](http://127.0.0.1:8000/api/docs)

شما باید یک صفحه زیبا با عنوان "ACRON Project API" ببینید که تمام Endpoint های مربوط به JWT (دریافت و رفرش توکن)، Customers (مشاهده و ویرایش پروفایل) و Products (لیست محصولات) در آن به دقت دسته بندی شده اند.

62- لطفاً این صفحه را بررسی کنید و ببینید آیا فیلدهای محصولات و تصاویر فرعی به درستی در مستندات نمایش داده شده اند؟

رسیدیم به بخش جذاب مستندسازی حرفه ای! دکوراتور `extend_schema@` دقیقاً همان ابزاری است که تفاوت یک API ساده را با یک API تجاری و استاندارد (مثل API های شرکت های بزرگ) مشخص می کند.

گاهی اوقات `drf-spectacular` با تمام هوشمندی اش نمی تواند هدف دقیق یک API را حدس بزند. مثلاً نمی داند که این API قرار است چه ارورهای خاصی برگرداند یا توضیح انسانی آن برای تیم فرانت اند چیست. اینجا است که `extend_schema@` وارد عمل می شود و کنترل کامل Swagger را به دست شما می سپارد.

معرفی و پیاده سازی `extend_schema@`

بیاید این جادو را روی همان `ProductListView` که در مرحله قبل ساختیم پیاده کنیم تا خروجی Swagger آن را از یک حالت رباتی به یک حالت کاملاً خوانا و حرفه ای تبدیل کنیم.

63- فایل `apps/products/views.py` را باز کنید و آن را به شکل زیر تغییر دهید:

```
from drf_spectacular.utils import extend_sch
ema, extend_schema_view
```

```
# استفاده از دکوراتور برای شخصی سازی مستندات
این View
extend_schema_view@
```

```

)get=extend_schema
summary="دریافت لیست محصولات فروشگاه",
description="این متد لیست تمامی محصولات را به همراه اطلاعات برنند، دسته بندی و گالری تصاویر برمیگرداند. این مسیر کاملاً بهینه سازی شده (بدون مشکل N+1) است و نیازی به توکن احراز هویت ندارد.",
tags=['Products Catalog']
# دسته بندی
ی API در ساید بار Swagger
(
(

```



کالبدشکافی کد (بدون Vibe Coding)

- استفاده (مثل `ListAPIView`) Generic چون ما از کلاس های `@extend_schema_view` به صورت مخفی در پس زمینه وجود دارد. این دکوراتور به ما اجازه `get` می کنیم، متد می دهد به متدهای مخفی کلاس دسترسی پیدا کنیم و به جنگو بگوییم: "تنظیماتی که این کلاس اعمال کن `get` می گویم را فقط روی متد".
- با فونت درشت Swagger یک عنوان کوتاه و جذاب که در لیست کشویی `summary` نوشته می شود API روبروی مسیر.
- کلیک می کند، آن را API توضیحات تکمیلی که وقتی فرانت اند کار روی `description` می خواند تا دقیقاً بفهمد این مسیر چه دیتایی به او می دهد.
- ها به صورت پیش فرض در یک گروه قرار می گیرند که API تمام، Swagger، در `tags` در صفحه `Products Catalog` شلوغ می شود. با تگ گذاری، یک پوشه (گروه) جدید به نام با نظم خاصی زیرمجموعه آن قرار می گیرد API ساخته می شود و این.



آیا به آموزش بیشتر Swagger در این مرحله نیاز داریم؟

پاسخ کوتاه: فعلاً خیر. شما اکنون ابزار اصلی (`drf-spectacular`) و ابزار شخصی سازی (`@extend_schema`) را در اختیار دارید. این ترکیب برای ۹۵٪ نیازهای مستندسازی یک پروژه در

این سطح کافی است.

درگیر شدن بیشتر با جزئیات Swagger (مثل تعریف دستی پارامترهای Query یا Override کردن کامپوننت‌های امنیتی) در این لحظه، باعث می‌شود از هدف اصلی باز بمانید. مستندسازی باید همگام با توسعه پیش برود، نه اینکه مانع آن شود.



بازگشت به مسیر توسعه: تکمیل فاز ۴ (Product Domain)

در فاز ۴، ما لیست محصولات (`ProductListView`) را ساختیم. اما در هر فروشگاه، وقتی روی یک محصول کلیک می‌کنید، وارد صفحه جزئیات آن محصول (**Product Detail**) می‌شوید. در اینجا به جای استفاده از `id` محصول (مثل `/api/products/1/`)، باید از `slug` استفاده کنیم (مثل `/api/products/samsung-galaxy-s24/`) تا URL‌ها برای سئو (SEO) بهینه باشند.

64- فایل `apps/products/views.py` را باز کنید و ویوی جزئیات را به انتهای آن اضافه کنید:

```
from rest_framework.generics import RetrieveAPIView
```

```
)extend_schema_view@
)get=extend_schema
summary="دریافت جزئیات یک محصول خاص",
description="این مسیر اطلاعات کامل یک محصول را بر اساس Slug آن برمی‌گرداند.",
tags=['Products Catalog']
(
(
:class ProductDetailView(RetrieveAPIView)
permission_classes = [AllowAny]
serializer_class = ProductSerializer
```

استفاده از همان تکنیک بهینه‌سازی دیتابیس

س

```
queryset = Product.objects.select_related()  
)d  
, 'category'  
'brand'  
)prefetch_related.(  
'media_gallery'  
()all.(
```

جادوی جنگو: جستجو بر اساس فیلد slug به جای id پیشفرض

```
'lookup_field' = 'slug'
```

65- فایل `apps/products/urls.py` را باز کنید و مسیر جدید را اضافه کنید:

```
from django.urls import path  
from .views import ProductListView, ProductDetail  
etailView
```

```
] = urlpatterns  
path('', ProductListView.as_view(), name  
,='product-list')
```

مسیر دریافت یک محصول بر اساس اسلاگ

```
path('<slug:slug>/', ProductDetailView.a  
,s_view(), name='product-detail')  
[
```

کالبدشکافی معماری این بخش

- `RetrieveAPIView`: مخصوص برگرداندن فقط یک رکورد از دیتابیس، DRF این کلاس آماده است.
- `lookup_field = 'slug'`: (Primary Key) `pk` ها دنبال URL در DRF، به صورت پیش فرض: "از این به DRF می‌گردد. با نوشتن این یک خط کد، به ID یا همان `id` در دیتابیس پیدا کن، نه `slug` بعد، محصول را از طریق فیلد".
- به جنگو می‌گویند که این پارامتر می‌تواند (`slug:`) بخش اول در `<slug:slug>/` URL نام متغیری است که (`slug`) شامل حروف، اعداد، خط تیره و آندرلاین باشد. بخش دوم پاس داده می‌شود در View در `lookup_field` به اکون فاز ۴ (Product Domain) به یک پایداری بسیار عالی رسیده است. هم مدل‌های بهینه داریم، هم ادمین قدرتمند، هم API‌های بدون مشکل N+1 و هم مستندات تمیز.



یکی از مهم‌ترین اصول مهندسی نرم‌افزار یعنی **DRY (Don't Repeat Yourself)** یا "خودت را تکرار نکن"



ریفکتور فاز ۴: استفاده از ViewSet برای جلوگیری از تکرار

66- فایل `apps/products/views.py` را باز کنید و کل کدهای قبلی (مربوط به محصولات) را با این کد جایگزین کنید:

```
from rest_framework.viewsets import ReadOnlyModelViewSet
from rest_framework.permissions import AllowAny
from drf_spectacular.utils import extend_schema, extend_schema_view
from .models import Product
from .serializers import ProductSerializer

# تعریف Swagger فقط یکبار در بالای ViewSet
```

```

)extend_schema_view@
)list=extend_schema
,"دریافت لیست محصولات"=summary
,"لیست تمامی محصولات به هم
راه برند، دسته بندی و گالری تصاویر."
,tags=['Products Catalog']
,(
)retrieve=extend_schema
,"دریافت جزئیات محصول"=summary
,"اطلاعات کامل یک محصول بر
اساس Slug"
,tags=['Products Catalog']
(
(
:class ProductViewSet(ReadOnlyModelViewSet)
permission_classes = [AllowAny]
serializer_class = ProductSerializer
'lookup_field = 'slug

queryset = Product.objects.select_related
)d
,'category'
'brand'
)prefetch_related.(
'media_gallery'
)all.(

```

چیستی و چرایی این تغییر:

- برای نمایش `ListAPIView` چیست؟ این کلاس جادویی ترکیبی از `ReadOnlyModelViewSet` و `RetrieveAPIView` (برای نمایش یکی) و (همه است. چون محصولات قرار نیست توسط

استفاده کردیم `ReadOnly` شوند، از نسخه (`DELETE`) یا پاک (`POST`) کاربران عادی ساخته تا امنیت حفظ شود.

- چرا `extend_schema_view@` تغییر کرد؟ به جای اینکه دکوراتور را روی دو کلاس مختلف بنویسیم، آن را یک بار بالای `ViewSet` نوشتیم. کلمه `list` به متد نمایش همه، و کلمه `retrieve` به متد نمایش جزئیات اشاره می‌کند.

67- چون از `ViewSet` استفاده کردیم، فایل `apps/products/urls.py` هم باید تغییر کند. آن را باز کنید و با این کد جایگزین کنید:

```
from rest_framework.routers import DefaultRouter
from .views import ProductViewSet

# استفاده از روتر برای تولید خودکار URLها
router = DefaultRouter()
router.register('', ProductViewSet, basename='product')

urlpatterns = router.urls
```

چرایی این تغییر:

- به آن‌ها `ViewSet` چیست؟ روترها ابزارهای هوشمندی هستند که وقتی یک `DefaultRouter` می‌دهید، خودشان می‌فهمند که باید مسیر `/` را برای لیست محصولات و مسیر `<slug:slug>/` را برای جزئیات محصول بسازند. این یعنی کدهای `urls.py` به شدت خلوت و تمیز می‌شوند.



این قسمت اختیاری هست

برای ایجاد یک نقطه ورود (API Root) که تمام مسیرها را به صورت لینک‌های قابل کلیک در مرورگر نمایش دهد، Django REST Framework (DRF) یک قابلیت عالی به نام `Browsable API` دارد.

برای پیاده‌سازی این ویژگی، بهترین روش استفاده از یک `api_view` است. شما درخواست کردید که از `Serializer` هم استفاده شود؛ اگرچه برای مسیرهای ثابت (Static URLs) معمولاً نیازی به سریالایزر نیست و دیکشنری‌های ساده پایتون کفایت می‌کنند، اما من کد را طوری تنظیم می‌کنم که هم از سریالایزر عبور کند (برای حفظ ساختار یکپارچه) و هم دسترسی آن برای همه آزاد (Public) باشد.

در ادامه مراحل تغییرات در دو فایل `urls.py` و `views.py` آورده شده است.

ویرایش فایل `urls.py`

ابتدا باید برای مسیرهای خود پارامتر `name` تعریف کنید تا بتوانیم در صورت نیاز به آن‌ها ارجاع دهیم و مسیر اصلی (root) را به یک `view` متصل کنیم.

```
from django.urls import include, path
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)
from . import views

urlpatterns = [
    # 📌 ها را نشان می‌دهد API مسیر اصلی که لیست تمام
    path('', views.api_root_view, name='api-root'),

    #....
]
```

. اضافه کردن ویو و سریالایزر در `views.py`

حالا در فایل `project/apps/api/views.py`، یک سریالایزر ساده برای تعیین ساختار خروجی می‌سازیم و سپس ویوی `api_root_view` را با دسترسی آزاد (`AllowAny`) ایجاد می‌کنیم.

برای اینکه لینک‌ها در مرورگر به صورت کامل (Absolute URI) و قابل کلیک نمایش داده شوند، از متد `request.build_absolute_uri()` استفاده می‌کنیم. این روش بسیار امن است زیرا به نام‌گذاری (namespacing) داخل فایل‌های `include` شده وابستگی ندارد.

```
from rest_framework.permissions import AllowAny
from rest_framework import serializers as rest_serializers

# --- View ---
```

```

@api_view(['GET'])
@permission_classes([AllowAny]) # 🔓 دسترسی آزاد برای همه کاربران
def api_root_view(request):
    """
    که مسیرهای موجود را لیست میکند API نقطه ورود
    """
    # ساخت دیتای خام شامل لینکهای قابل کلیک
    raw_data = {
        "authentication": {
            "token": request.build_absolute_uri('token/'),
            "token_refresh": request.build_absolute_uri('token/refresh/'),
        },
        "user_management": {
            "me": request.build_absolute_uri('me/'),
        },
        "resources": {
            "customers": request.build_absolute_uri('customers/'),
            "products": request.build_absolute_uri('products/'),
        }
    }

    # عبور دادن دیتا از سریالایزر (طبق درخواست شما)
    serializer = APIDirectorySerializer(data=raw_data)
    serializer.is_valid(raise_exception=True)

    # به صورت گرافیکی رندر میشو DRF بازگرداندن پاسخ که توسط
    return Response(serializer.validated_data)

```

مزایای این روش:

- **رابط کاربری DRF:** وقتی این مسیر `/http://your-domain/api` را در مرورگر باز کنید، پِنل زیبای Django REST Framework به شما یک فایل JSON قالب‌بندی شده به همراه

لینک‌های آبی‌رنگ و قابل کلیک برای هر بخش (احراز هویت، محصولات و...) نمایش می‌دهد.

- **توسعه‌پذیری:** دسته‌بندی لینک‌ها در دیکشنری (مثل `resources` و `authentication`) به تیم فرانت‌اند کمک می‌کند ساختار API را بهتر درک کنند.

دلیل اینکه در مرورگر صفحه سفید می‌بینید، به احتمال خیلی زیاد به سمت کلاينت (مرورگر) برمی‌گردد. در پروژه‌های جنگو، این اتفاق معمولاً به چند دلیل مشخص رخ می‌دهد:

۱. تداخل Django Debug Toolbar با DRF (محتمل‌ترین دلیل)

در لاگ‌های شما مشخص است که `debug_toolbar` در حال لود شدن است (GET). گاهی اوقات اسکریپت‌ها و استایل‌های Debug Toolbar با رابط کاربری DRF تداخل پیدا می‌کنند (مخصوصاً بر سر نسخه jQuery) و باعث می‌شوند رندر صفحه در مرورگر متوقف شود و یک صفحه سفید ببینید.

- **راه حل تست:** برای چند لحظه در فایل `settings.py`، میدل‌ور (Middleware) مربوط به `debug_toolbar` را کامنت کنید و سرور را ری‌استارت کنید. اگر صفحه DRF نمایش داده شد، مشکل از همین تداخل است.

۲. افزونه‌های مرورگر (AdBlockers یا JSON Viewers)

بسیاری از مسدودکننده‌های تبلیغات (مثل uBlock Origin یا AdBlock) وقتی کلمه `api` را در URL می‌بینند، به صورت پیش‌فرض درخواست‌ها را بلاک کرده یا تگ‌های صفحه را پنهان می‌کنند. همچنین اگر افزونه‌ای برای زیباسازی (JSON Formatter) JSON نصب کرده باشید، ممکن است سعی کند صفحه HTML تولید شده توسط DRF را به عنوان JSON پارس کند و در نتیجه کرش کند و صفحه سفید شود.

- **راه حل تست:** مسیر را در حالت ناشناس مرورگر (Incognito/Private Window) باز کنید.

۳. بررسی دیتای خام با پارامتر format

برای اینکه خیالتان راحت شود که دیتای سریالایزر به درستی در حال ارسال است و مشکل فقط از قالب گرافیکی DRF است، به انتهای URL خود در مرورگر این عبارت را اضافه کنید:

<http://127.0.0.1:8000/api/?format=json>

با این کار DRF متوجه می‌شود که شما صفحه HTML را نمی‌خواهید و فقط دیتای خالص JSON را به شما برمی‌گرداند. اگر با این کار دیتا را دیدید، کدهای `views.py` شما کاملاً بی‌نقص هستند.

۴. کنسول مرورگر (Developer Tools)

کلید **F12** را در مرورگر بزنید و به تب **Console** بروید. اگر صفحه به دلیل خطاهای جاوااسکریپتی (مثل لود نشدن صحیح یک فایل یا تداخل متغیرها) سفید شده باشد، خطاها را با رنگ قرمز در اینجا خواهید دید.

پایان Part-5